# Progettazione di una architettura multi-agente RAG per il supporto alla centrale di AREU (112)

Andrea De Tomasi

Aprile 2026

# Abstract

L'integrazione dei modelli di linguaggio di grandi dimensioni (LLM) in settori critici è spesso ostacolata da limiti strutturali quali le *allucinazioni* e la staticità della conoscenza. Il presente lavoro descrive la progettazione e l'implementazione di un'architettura multi-agente di Retrieval-Augmented Generation(RAG) finalizzata a supportare gli operatori della centrale dell' *Agenzia Regionale Emergenza Urgenza* (AREU) nella consultazione rapida di procedure operative e protocolli medici complessi.

Il sistema è orchestrato utilizzando LangGraph in combinazione con DecAgents, una libreria proprietaria di Ottante S.r.l. (parte del gruppo Beta 80). Sfruttando le funzionalità di base e le capacità di networking di DecAgents — una libreria già distribuita in diversi ambienti di produzione basati su cloud — l'architettura supera la linearità delle pipeline tradizionali grazie all'impiego di un grafo computazionale, nel quale agenti autonomi specializzati collaborano alla decomposizione delle query, al recupero semantico delle informazioni e alla validazione dei risultati. Per garantire la massima precisione nel retrieval, è stata adottata una strategia di "query expansion" basata su "hypothetical queries" e un sistema di reranking basato su una funzione sigmoide, volto a minimizzare il rumore informativo. L'intera infrastruttura è ospitata su Amazon Bedrock, garantendo elevati standard di sicurezza e riservatezza nel trattamento dei dati sensibili dell'Agenzia.

I test condotti con il coinvolgimento diretto del personale di AREU dimostrano che l'integrazione di meccanismi di reasoning per ogni nodo del grafo aumenta significativamente la trasparenza e l'affidabilità del sistema. Sebbene i modelli testati (Amazon Nova-2-Lite e Claude Sonnet) abbiano mostrato punteggi di soddisfazione analoghi, l'analisi delle prestazioni ha evidenziato la latenza come fattore critico per l'operatività in emergenza, identificando nel filtraggio dei documenti il principale collo di bottiglia. In conclusione, il lavoro dimostra come un approccio multi-agente possa trasformare la documentazione aziendale in un dialogo intelligente e sicuro, offrendo un supporto decisionale concreto in situazioni ad alto stress operativo.

# Table of Contents

Abstract . . . 2
1 Introduzione . . . 4
1.1 Il ruolo di AREU e la gestione del 118 . . . 4
1.2 La necessità di un assistente intelligente . . . 4
1.3 Impatto operativo e visione futura . . . 5
2 Stato dell'Arte: Sistemi di Retrieval-Augmented Generation (RAG) . . . 6
2.1 Genesi del Paradigma RAG . . . 6
2.2 Architettura e Componenti di un Sistema RAG . . . 6
2.3 Evoluzione dei paradigmi RAG . . . 8
2.4 Frontiere agentiche, valutazioni e sfide future . . . 10
2.5 Conclusione . . . 11
3 Metodi . . . 12
3.1 Architettura del Sistema . . . 12
3.1.1 Selezione del database e dei modelli . . . 12
3.1.2 Frameworks di Orchestrazione: LangGraph e DecAgents . . . 12
3.1.3 Descrizione pipeline . . . 14
3.2 Dataset e Preprocessing . . . 15
3.3 Retrieval con reasoning . . . 16
3.4 Generazione e Validazione . . . 17
4 Risultati . . . 19
5 Discussione . . . 22
Bibliography . . . 24
Appendice . . . 25

# 1 Introduzione

L'integrazione dell'Intelligenza Artificiale nei settori critici rappresenta una delle sfide più ambiziose dell'era moderna. Sebbene i modelli di linguaggio di grandi dimensioni (LLM) abbiano dimostrato capacità straordinarie nel comprendere e generare testi, essi portano con sé limiti strutturali non trascurabili, come la tendenza a generare informazioni errate (le cosiddette *allucinazioni*) e il possesso di una conoscenza statica, limitata al momento del loro addestramento. In un ambito dove l'accuratezza dell'informazione può fare la differenza tra il successo e il fallimento di un intervento, questi limiti non sono accettabili.

## 1.1 Il ruolo di AREU e la gestione del 118

Il presente lavoro si colloca nel cuore pulsante del sistema di soccorso lombardo: l'*Agenzia Regionale Emergenza Urgenza* (AREU), l'ente che coordina il Numero Unico di Emergenza 118. Operare in una centrale di emergenza significa gestire situazioni ad altissima pressione, dove la rapidità e l'affidabilità delle informazioni sono cruciali. Gli operatori devono avere accesso immediato a un vasto patrimonio di conoscenze che include istruzioni operative, procedure interne e protocolli medici complessi, spesso contenuti in documenti densi e non sempre facili da consultare rapidamente durante una chiamata.

## 1.2 La necessità di un assistente intelligente

L'obiettivo di questo progetto è la creazione di una architettura multi-agente basata sul paradigma Retrieval-Augmented Generation (RAG), progettato specificamente per supportare il personale della centrale. A differenza di una normale IA, questo sistema non si affida solo alla propria "memoria" interna, ma agisce come un bibliotecario esperto: alla ricezione di una domanda, interroga una memoria esterna composta dai documenti ufficiali di AREU, estrae le informazioni pertinenti e le rielabora per fornire una risposta chiara e verificabile. Questo approccio garantisce che ogni suggerimento fornito dall'agente sia ancorato a fonti attendibili e aggiornate, riducendo drasticamente il rischio di errori. Inoltre, la scelta di utilizzare infrastrutture avanzate come Amazon Bedrock non è solo tecnica, ma strategica: essa permette di gestire dati sensibili garantendo il massimo controllo e la riservatezza dei documenti, un requisito imprescindibile quando si trattano procedure di soccorso e protocolli interni.

## 1.3 Impatto operativo e visione futura

Il sistema non nasce in isolamento, ma è il frutto di un processo che ha visto il coinvolgimento diretto degli operatori di AREU, i quali hanno testato la pertinenza delle risposte e la facilità d'uso dell'interfaccia. L'idea alla base è quella di offrire uno strumento di supporto decisionale che possa servire a diverse figure professionali: dai soccorritori sul campo agli operatori sanitari, fino ai responsabili della logistica. In sintesi, l'introduzione di questa tecnologia in AREU mira a trasformare il modo in cui le informazioni vengono reperite in emergenza, passando da una ricerca manuale e potenzialmente lenta a un dialogo intelligente con la documentazione aziendale, contribuendo così a rendere le decisioni più rapide, informate e sicure per la cittadinanza.

# 2 Stato dell'Arte: Sistemi di Retrieval-Augmented Generation (RAG)

## 2.1 Genesi del Paradigma RAG

Negli ultimi anni, i Large Language Models (LLM) hanno dimostrato elevate capacità di apprendere conoscenza fattuale e semantica da grandi quantità di dati testuali. Tuttavia, tali modelli presentano limiti strutturali significativi, tra cui la difficoltà di aggiornamento continuo, la scarsa interpretabilità e la tendenza a generare informazioni errate o non verificabili, fenomeno noto come "generation of hallucination". Queste limitazioni derivano dalla natura parametrica della conoscenza incorporata nei modelli: essa è codificata nei pesi come risultato di un processo di ottimizzazione su larga scala e rimane sostanzialmente statica.

Dal punto di vista matematico, tale conoscenza corrisponde a un punto (o a una regione) di minimo della funzione di perdita nello spazio dei parametri; modificarla in modo mirato richiede quindi una nuova fase di addestramento o una ri-ottimizzazione, poiché non è possibile intervenire localmente sui singoli contenuti senza alterare l'equilibrio globale dei parametri appresi.

Per affrontare tali problematiche, il paradigma della Retrieval-Augmented Generation (RAG) è stato introdotto da Lewis et al. nel 2020 [Lew+20]. In questo approccio, la conoscenza parametrica del modello viene affiancata da una memoria esterna non parametrica, composta da collezioni di documenti preventivamente indicizzati.

## 2.2 Architettura e Componenti di un Sistema RAG

Un sistema RAG si basa su tre componenti fondamentali:

- retrieval da fonti esterne
- integrazione della conoscenza
- generazione del testo

L'accesso alle fonti esterne di conoscenza, avviene tramite tecniche di retrieval semantico, vale a dire metodi che non si limitano alla semplice corrispondenza lessicale tra parole

chiave, ma rappresentano query e documenti come vettori in uno spazio semantico continuo (embedding). In questo spazio vettoriale, la similarità tra testi viene calcolata sulla base della distanza o dell'angolo tra i rispettivi vettori (ad esempio tramite similarità coseno), consentendo di recuperare documenti rilevanti anche quando non condividono esattamente gli stessi termini della query, ma ne esprimono il contenuto concettuale. In tal modo, il modello può integrare dinamicamente informazioni aggiornate o specialistiche senza dover modificare i propri parametri interni. Lo strumento tipicamente usato per questa fase di un sistema RAG è il retriever neurale, un sistema composto da uno o più encoder neurali (tipicamente basati su architetture Transformer) addestrati a mappare sia le query sia i documenti in vettori densi appartenenti a uno spazio semantico condiviso. Nella configurazione più comune, detta bi-encoder, un encoder codifica la query e un altro codifica i documenti (talvolta con pesi condivisi). Ciascun testo viene così mappato in un vettore $f(x) \in \mathbb{R}^d$, dove la dimensione $d$ è fissata a priori.

L'architettura Transformer, introdotta da Vaswani et al. (2017) [Vas+17], si basa esclusivamente su meccanismi di attenzione, eliminando completamente ricorrenze e convoluzioni. Il componente centrale è la self-attention, che consente a ciascun elemento di una sequenza di pesare dinamicamente tutti gli altri elementi, modellando dipendenze a lungo raggio in modo parallelo. Dato un insieme di query $Q$, key $K$ e value $V$, l'attenzione scalata (scaled dot-product attention) è definita come:

$$\mathrm{Attention}(Q, K, V) = \mathrm{Softmax}\left(\frac{QK^{\mathbb{T}}}{\sqrt{d_k}}\right)V$$

dove $d_k$ è la dimensione delle key. Il fattore di normalizzazione $\frac{1}{\sqrt{d_k}}$ stabilizza il gradiente per grandi dimensioni vettoriali.

Invece di applicare una singola funzione di attenzione con key, value e query di dimensione $d_{\text{model}}$, risulta vantaggioso proiettare linearmente le query, le key e le value $h$ volte tramite diverse proiezioni lineari apprese, rispettivamente verso spazi di dimensione $d_k$, $d_k$ e $d_v$. La multi-head attention consente al modello di prestare attenzione congiuntamente a informazioni provenienti da diversi sottospazi di rappresentazione in posizioni differenti. Con una singola testa di attenzione, l'operazione di media tende a inibire questa capacità:

$$\mathrm{MultiHead}(Q, K, V) = \mathrm{Concat}(\mathrm{head}_1, ..., \mathrm{head}_h)W^O$$

$$\text{dove} \quad \mathrm{head}_i = \mathrm{Attention}\left(QW_i^Q, KW_i^K, VW_i^V\right)$$

e $W_i^Q, W_i^K, W_i^V$ e $W^O$ sono matrici parametriche apprese.

Grazie a questa architettura, il Transformer consente un'elaborazione interamente parallela delle sequenze e una modellazione efficiente delle dipendenze globali, ponendo le basi per i moderni modelli di linguaggio di grandi dimensioni.

La loss function è il criterio matematico che guida l'addestramento del retriever. In genere è di tipo contrastivo: dato un insieme di coppie query-documento rilevanti (positivi) e non rilevanti (negativi), la perdita è definita in modo da massimizzare la similarità tra i vettori delle coppie positive e minimizzare quella delle coppie negative. Formalmente, si ottimizzano i parametri dell'encoder affinché la struttura geometrica dello spazio vettoriale rifletta una nozione di rilevanza semantica. In questo modo, la funzione appresa induce una metrica nello spazio degli embedding che approssima il concetto di pertinenza rispetto alla query.

I documenti più rilevanti individuati dal retriever vengono forniti come contesto al modello generativo, che li utilizza per produrre la risposta. Questa integrazione tra recupero semantico e generazione testuale consente di arricchire la conoscenza parametrica del modello con informazioni esterne aggiornabili, migliorando le prestazioni nei compiti ad alta intensità di conoscenza.

Questo paradigma ha dimostrato di aumentare la fattualità, la trasparenza e l'aggiornabilità dei sistemi basati su LLM, riducendo al contempo la necessità di riaddestramento continuo. Inoltre, la possibilità di tracciare le fonti informative durante la generazione rappresenta un elemento chiave per la fiducia e la verificabilità delle risposte [Gao+23].

## 2.3 Evoluzione dei paradigmi RAG

La letteratura recente evidenzia una progressiva evoluzione delle architetture RAG, caratterizzata da crescente modularità e da un'integrazione sempre più stretta tra retrieval e generazione. Le prime implementazioni seguivano una pipeline lineare, in cui una query veniva trasformata in embedding, utilizzata per recuperare documenti e successivamente fornita al modello generativo [Gao+23].

In questa fase iniziale, i meccanismi di retrieval derivavano in larga parte dall'Information Retrieval tradizionale: approcci lessicali come BM25 [Rob+94] sono stati progressivamente affiancati e poi sostituiti da modelli neurali densi, basati su rappresentazioni vettoriali condivise tra query e documenti. L'adozione di embedding semantici ha migliorato la robustezza e la capacità di generalizzazione dei sistemi.

Con la maturazione del paradigma, la ricerca ha introdotto miglioramenti modulari volti a ottimizzare la qualità del contesto fornito al generatore. Tra questi rientrano strategie di chunking dei documenti (segmentazione in unità informative coerenti), query expansion, selezione dinamica del contesto, re-ranking e filtraggio semantico. Tali tecniche riflettono la consapevolezza che la nozione di rilevanza nei sistemi generativi differisce da quella dei sistemi di retrieval tradizionali [Che+25].

Un'evoluzione ulteriore riguarda l'integrazione tra retrieval e reasoning. Le architetture moderne includono approcci multi-hop, in cui il sistema esegue più iterazioni di recupero e ragionamento, nonché strategie di decomposizione della query. Questo consente di affrontare task complessi che richiedono inferenze distribuite su più documenti o fonti eterogenee.

In tale prospettiva, un aspetto centrale diventa l'allineamento tra retriever e generatore. Zhou & Chen (2025) propongono un'ottimizzazione end-to-end del retriever basata sulla rilevanza in-context, dimostrando che la rilevanza per il contesto generativo differisce da quella dei sistemi di retrieval tradizionali [ZC25]. Questo approccio evidenzia come l'efficienza del retrieval possa incidere significativamente sulle prestazioni complessive, anche in presenza di modelli generativi di dimensioni contenute.

Parallelamente, studi recenti sottolineano l'importanza della gestione della finestra di contesto degli LLM. Tecniche di compressione semantica, selezione adattiva dei documenti e controllo del rumore mirano a ottimizzare l'uso del contesto disponibile. In particolare, Cuconasu et al. (2024) mostrano che un filtraggio e una ponderazione appropriata dei documenti recuperati riducono il rumore e migliorano l'efficacia del modello [Cuc+24], mentre Cheng et al. (2025) forniscono una panoramica sistematica delle strategie più recenti per combinare e selezionare i documenti in modo efficiente [Che+25].

Un risultato controintuitivo emerso in letteratura riguarda il ruolo del rumore nel processo di retrieval. Cuconasu et al. (2024) mostrano che l'introduzione controllata di documenti non perfettamente rilevanti può migliorare l'accuratezza del modello generativo, suggerendo che un contesto più diversificato favorisca il processo inferenziale [Cuc+24]. Una possibile congettura teorica a sostegno di tale risultato è che la presenza di informazione moderatamente eterogenea agisca come forma di regularizzazione contestuale: l'inclusione di documenti non strettamente sovrapponibili alla query potrebbe ridurre il rischio di sovra-adattamento del modello a evidenze parziali o ridondanti, favorendo una rappresentazione latente più robusta. Inoltre, un contesto diversificato potrebbe ampliare lo spazio semantico

esplorato dal modello durante la generazione, aumentando la probabilità di attivare connessioni concettuali pertinenti ma non immediatamente evidenti.

Un'ulteriore ipotesi è che il "rumore controllato" migliori la calibrazione dell'attenzione: la necessità di discriminare tra segnali fortemente e debolmente rilevanti potrebbe indurre il modello a pesare in modo più selettivo le informazioni disponibili, rafforzando i meccanismi di integrazione e confronto tra fonti. In questa prospettiva, l'effetto positivo osservato non deriverebbe dal rumore in sé, ma dalla sua funzione di stimolo alla selezione e composizione inferenziale all'interno della finestra di contesto. Questo risultato mette in discussione le metriche tradizionali di valutazione del retrieval e apre nuove prospettive nella progettazione dei sistemi RAG.

Infine, si osserva un crescente interesse per l'integrazione con sistemi multimodali, database strutturati e grafi di conoscenza, ampliando il ruolo del retrieval oltre il semplice accesso a documenti testuali e configurando i sistemi RAG come infrastrutture ibride per l'accesso e la composizione della conoscenza.

## 2.4 Frontiere agentiche, valutazioni e sfide future

Un'ulteriore direzione di ricerca riguarda l'efficienza computazionale e la scalabilità. L'adozione di architetture distribuite, indicizzazione incrementale e caching semantico consente di applicare i sistemi RAG in contesti reali, caratterizzati da vincoli di latenza e costi. Inoltre, l'integrazione con sistemi agentici e workflow automatizzati rappresenta un'area in rapida crescita. Per sistema agentico si intende un'architettura in cui il modello linguistico non si limita a generare una risposta a partire da un singolo prompt, ma opera come un agente capace di pianificare, prendere decisioni intermedie e interagire con strumenti esterni (ad esempio motori di ricerca, database, API o moduli di calcolo).

La frontiera più avanzata della ricerca si sta spostando verso i Multi-Agent Systems (MAS). In questo paradigma, la complessità del task non è affidata a un unico "agente generalista", ma viene distribuita tra una rete di agenti specializzati che collaborano tra loro. Questa scomposizione dei compiti permette di assegnare ruoli distinti a diverse entità logiche, come ad esempio:

- Agente Ricercatore: specializzato nell'espansione semantica della query (hyper queries) e nel recupero accurato dei chunk.
- Agente Critico o Validatore: incaricato di valutare la pertinenza delle fonti e la coerenza della risposta, innescando cicli di correzione se i criteri di qualità non sono soddisfatti.
- Agente Sintetizzatore: focalizzato sulla generazione finale del testo, garantendo che sia ancorato alle fonti e privo di allucinazioni.

Questa transizione verso il multi-agente giustifica teoricamente l'adozione di architetture basate su grafi computazionali, dove ogni nodo non è una semplice funzione, ma un agente con un proprio ambito di competenza. L'approccio MAS aumenta la modularità e l'interpretabilità del sistema: poiché ogni agente deve fornire un "reasoning" del proprio operato, diventa possibile tracciare con precisione dove avvenga un eventuale errore nella catena di ragionamento.

Nonostante i progressi nella definizione e nell'implementazione di sistemi RAG efficienti ed esaustivi, permangono ancora — tra le altre — le seguenti sfide::

- allineamento tra retrieval e generazione
- selezione efficiente della conoscenza
- interpretabilità dei sistemi
- valutazione realistica delle prestazioni
- gestione di informazioni obsolete o contraddittorie
- definizione di benchmark realistici e metriche di valutazione adeguate

## 2.5 Conclusione

Il paradigma RAG si è evoluto da un semplice sistema di recupero documentale a un ecosistema complesso, modulare e orientato alla conoscenza. L'integrazione dinamica della conoscenza esterna consente di migliorare l'affidabilità, la fattualità e la trasparenza dei modelli linguistici, rendendo tali sistemi sempre più adatti ad applicazioni reali in domini specialistici.

Le direzioni future includono sistemi multimodali, agenti autonomi, retrieval adattivo, integrazione con grafi di conoscenza e strategie avanzate di reasoning. Queste innovazioni potrebbero portare allo sviluppo di sistemi intelligenti più robusti, interpretabili e aggiornabili.

# 3 Metodi

## 3.1 Architettura del Sistema

### 3.1.1 Selezione del database e dei modelli

Per l'implementazione del sistema è stata effettuata una selezione mirata sia del database utilizzato per la memorizzazione delle informazioni sia dei modelli impiegati nelle fasi di retrieval e generazione. In particolare, è stato scelto di utilizzare PostgreSQL come sistema di gestione del database, arricchito con pgvector, un'estensione che consente la memorizzazione e la ricerca efficiente di embedding vettoriali. Questa soluzione permette di integrare funzionalità di vector search direttamente all'interno di un database relazionale, semplificando l'infrastruttura del sistema e facilitando la gestione dei documenti e dei relativi embedding utilizzati nel processo di retrieval.

Per quanto riguarda i modelli di intelligenza artificiale, è stata utilizzato il servizio Amazon Bedrock, che fornisce accesso a diversi modelli fondamentali attraverso un'interfaccia unificata. L'impiego di questa piattaforma consente di integrare modelli per embedding e generazione, mantenendo un'architettura flessibile e scalabile e facilitando allo stesso tempo la sperimentazione e la sostituzione dei modelli senza modifiche sostanziali all'infrastruttura del sistema. Inoltre, l'impiego di Amazon Bedrock consente di gestire l'accesso ai modelli tramite l'infrastruttura Amazon Web Services, garantendo un maggiore controllo sui dati e contribuendo a preservare la riservatezza dei documenti utilizzati dal sistema, aspetto particolarmente rilevante nel contesto di questo lavoro.

### 3.1.2 Frameworks di Orchestrazione: LangGraph e DecAgents

Per l'implementazione e l'orchestrazione del workflow del sistema RAG è stato scelto di utilizzare LangGraph, una libreria sviluppata per la costruzione di applicazioni basate su Large Language Models attraverso strutture computazionali a grafo. La scelta viene dalla necessità di definire un processo di elaborazione articolato, che prevede integrazioni dinamiche tra diverse fasi, modularità e flessibilità nell'esecuzione, garantendo così la robustezza necessaria in un contesto critico come quello di AREU. LangGraph estende il paradigma delle pipeline lineari tipicamente utilizzate nei framework per LLM, consentendo di definire flussi di esecuzione più complessi caratterizzati da transizioni condizionali e cicli iterativi. L'implementazione, tuttavia, non si basa esclusivamente su LangGraph; integra anche le funzionalità di base e i livelli di networking di DecAgents. DecAgents è una libreria proprietaria sviluppata da Ottante S.r.l., azienda appartenente al gruppo Beta 80, progettata

per facilitare la creazione di sistemi multi-agente con particolare attenzione alla modularità, alla scalabilità e alla prontezza per l'utilizzo in ambienti di produzione.

A differenza delle architetture sequenziali tradizionali, nelle quali le operazioni vengono eseguite secondo una pipeline statica, LangGraph rappresenta il processo come un grafo diretto composto da nodi e archi. In questo contesto, ciascun nodo rappresenta un'unità computazionale — ad esempio un modulo di retrieval, reasoning o generazione — mentre gli archi definiscono il flusso di dati e le condizioni di transizione tra le diverse fasi del sistema. In pratica, ogni nodo è un agente autonomo che esegue una specifica funzione.

Un ulteriore vantaggio offerto da LangGraph è la possibilità di assegnare modelli diversi a ciascun nodo in base al tipo di task da eseguire. Questo significa che, a seconda della complessità e della criticità dell'operazione, è possibile utilizzare modelli più specializzati e performanti oppure versioni più leggere e meno costose. Ad esempio, task di semplice elaborazione o filtraggio dei dati possono essere affidati a modelli più economici, mentre la generazione di risposte complesse o il reasoning avanzato possono sfruttare modelli più potenti. Questa flessibilità consente un'ottimizzazione bilanciata tra costi computazionali e qualità delle prestazioni, adattando dinamicamente le risorse alle esigenze specifiche di ciascun nodo. Inoltre, facilita l'integrazione di nuovi modelli o aggiornamenti tecnologici senza la necessità di riprogettare l'intero workflow, preservando la modularità e la scalabilità dell'architettura.

Il processo di elaborazione è inoltre guidato da uno "stato" condiviso del grafo (graph state), ovvero una struttura dati che contiene le informazioni rilevanti accumulate durante l'esecuzione. Tale stato viene passato tra gli agenti e progressivamente aggiornato, permettendo al sistema di mantenere traccia del contesto corrente della richiesta e dei risultati intermedi prodotti dalle diverse componenti del workflow.

L'utilizzo di un'architettura basata su grafo ha consentito di implementare strategie di reasoning più articolate, come iterazioni tra retrieval e generazione, controlli di qualità sulle risposte prodotte o meccanismi di fallback quando le informazioni recuperate risultano insufficienti. Allo stesso tempo, tale struttura garantisce un'elevata modularità: ogni nodo del grafo incapsula una specifica funzionalità del sistema, permettendo di modificare o sostituire singoli componenti — come il retriever o il modello generativo — senza alterare il workflow complessivo. Queste caratteristiche rendono il sistema più flessibile, facilitando sia l'adattamento dinamico durante l'esecuzione sia la sperimentazione di diverse configurazioni architetturali.

### 3.1.3 Descrizione pipeline

Il workflow dell'architettura RAG è rappresentato nell'immagine Figure 1, che illustra tutte le fasi del processo. A seguito della ricezione di una query, un agente [`rag_or_not`] decide se terminare la conversazione [`end_conversation`], generare una risposta diretta senza recuperare i documenti [`direct_answer`], o procedere con il retrieval. Questa strategia consente di evitare il passaggio dalla fase di retrieval per le domande che non richiedono l'uso di documenti, ottimizzando così sia i costi computazionali sia i tempi di risposta.

In caso di retrieval, un altro agente [`generate_hyq`] genera due domande semanticamente vicine a quella originale (hypothetical queries), fa un embedding di tutte e tre le query e utilizza la media dei tre embedding per recuperare i documenti più rilevanti dal database [`retrieve_chunks_with_reasoning`]. I documenti recuperati vengono quindi forniti come contesto all'agente generativo [`gen_rag_answer`], che produce la risposta finale.

Il passo successivo consiste nella validazione della risposta generata [`validate_answer`], che viene confrontata con la domanda originale per verificare la coerenza e la correttezza delle informazioni fornite. In caso di esito negativo, il sistema può decidere di iterare nuovamente il processo di retrieval, aggiornando lo stato del grafo con le informazioni ottenute e generando nuove query per affinare ulteriormente la ricerca, oppure di terminare la conversazione se si ritiene che ulteriori tentativi non porteranno a un miglioramento significativo della risposta (ad esempio per mancanza di documentazione sufficiente [`fallback_message`]).

Qualora la risposta venga ritenuta soddisfacente, un agente genera un TL;DR [`add_sources_tldr`] dopodichè, il sistema restituisce all'utente finale una risposta contestualizzata e basata su fonti esterne, migliorando così l'affidabilità e la pertinenza delle informazioni fornite rispetto a un modello generativo standalone.

Tutti ciò che viene generato durante il processo (query, documenti recuperati, risposte prodotte, tempo impiegato da ciascun nodo, ...) viene memorizzato nello stato del grafo, consentendo al sistema di mantenere una traccia completa dell'interazione e di utilizzare queste informazioni per eventuali iterazioni successive o per analisi post-hoc delle prestazioni del sistema. Inoltre tutti gli step sono affiancati da "reasoning", ovvero da una spiegazione testuale che descrive le decisioni prese dal sistema in ciascuna fase, migliorando così la trasparenza e l'interpretabilità del processo.

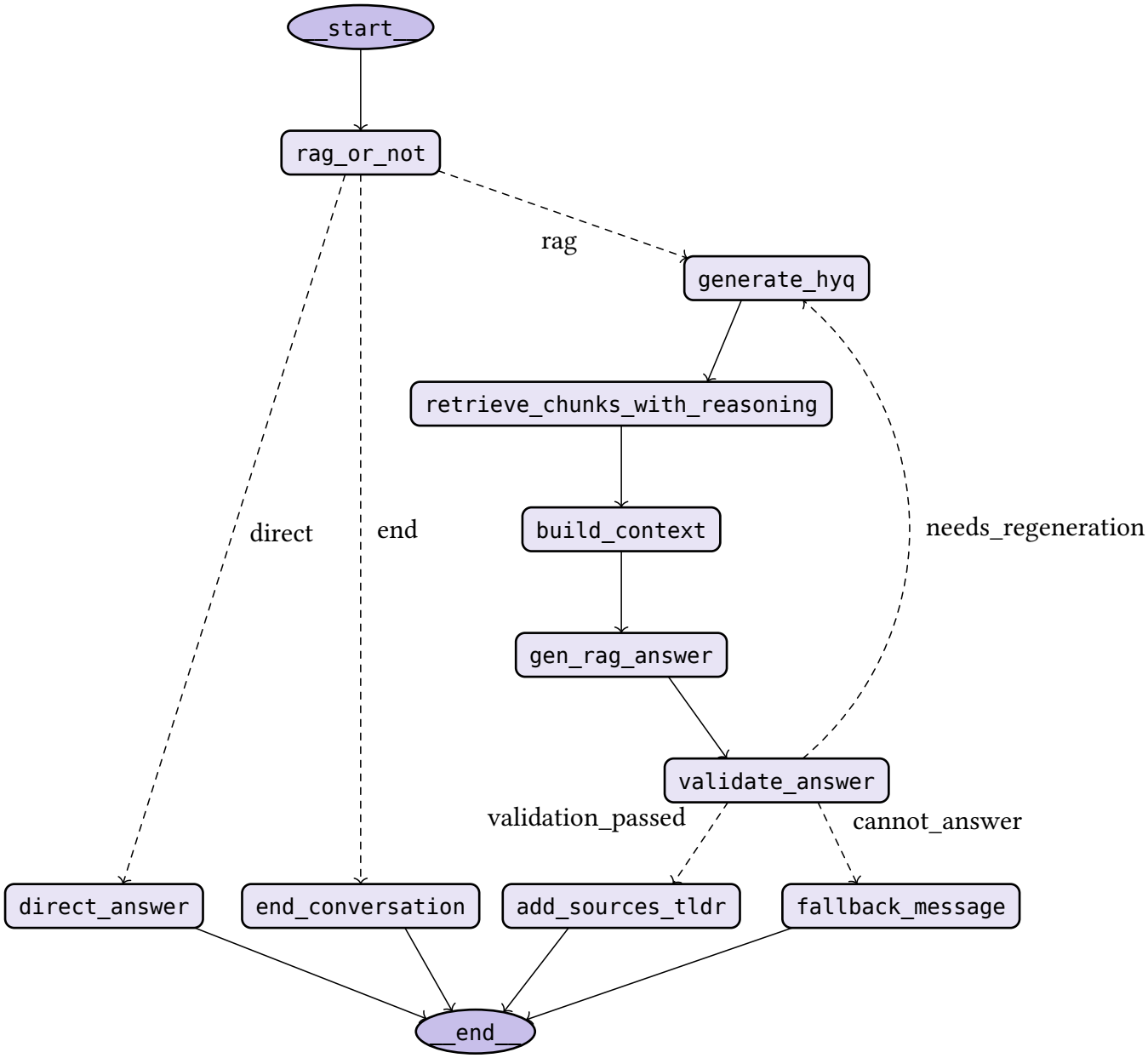


*Figure 1: RAG agent workflow*

## 3.2 Dataset e Preprocessing

I documenti utilizzati per il retrieval sono stati raccolti da fonti pubbliche e private, e includono istruzioni operative, procedure interne e documenti di rilievo per la gestione delle emergenze. Il processo di preprocessing ha coinvolto la pulizia dei dati, la normalizzazione del testo e la segmentazione in unità informative coerenti (chunking). Ogni "chunk" sufficientemente lungo viene affiancato da un "summary" e da tre "hypothetical queries", ovvero domande generate automaticamente le cui risposte sono contenute all'interno del chunk e che ne rappresentano diversi aspetti informativi, al fine di migliorare la copertura semantica durante il processo di retrieval. Si è osservato che avere delle "hypothetical

queries" affiancate al corpo testuale del chunk porta un notevole miglioramento della fase di retrieval. Tutti i chunk sono stati arricchiti con parole chiave prese dal contesto generale del documento, così da preservare anche quelle informazioni rilevanti che, pur essendo distribuite in punti diversi del testo, risultano fondamentali per la corretta comprensione del contenuto, assicurandosi che non vengano perse.

Il database è stato implementato utilizzando PostgreSQL con l'estensione pgvector, che consente di memorizzare e indicizzare efficientemente gli embedding vettoriali associati ai documenti. Gli embedding sono stati generati utilizzando Titan Text Embeddings V2, un modello di embedding disponibile su Amazon Bedrock, e memorizzati nel database insieme ai metadati dei documenti, come titolo e distinzione tra documenti pubblici e privati.

## 3.3 Retrieval con reasoning

Nella fase di retrieval, il sistema genera due hypothetical queries a partire dalla domanda dell'utente, le trasforma in embedding e utilizza la media dei tre embedding per recuperare i documenti più rilevanti dal database. Considerare i documenti più prossimi alla "media semantica" di tre domande tra loro simili consente di individuare con maggiore precisione i chunk realmente utili alla generazione della risposta. Successivamente, viene fatta una valutazione del contenuto dei documenti recuperati, con l'obiettivo di identificare la rilevanza e la pertinenza delle informazioni rispetto alla query originale. Questa valutazione viene fatta da un modello, il quale affianca ad ogni "chunk" un punteggio compreso tra 0 e 1, che rappresenta la probabilità che il documento sia rilevante per la query, insieme a una spiegazione testuale che descrive le motivazioni della valutazione.

Dopo questa fase, i documenti vengono ordinati sulla base della rilevanza e scremati in modo da mantenere solo quelli più pertinenti tramite una "sigmoide" centrata nella media dei punteggi con steepness 8, al fine di ridurre il rumore e migliorare la qualità del contesto fornito al modello generativo. Non è la rilevanza grezza a determinare il filtro: prima viene trasformata tramite la sigmoide. Di conseguenza, vengono mantenuti solo i documenti per cui il valore della sigmoide è superiore a 0.65, mentre quelli per cui la sigmoide restituisce un valore $\leq 0.65$ vengono scartati, come mostrato nella Figure 2. Una soluzione di "re-ranking" e "filtering" di questo tipo consente di scartare i chunk non rilevanti, senza correre il rischio di perdere informazioni utili.

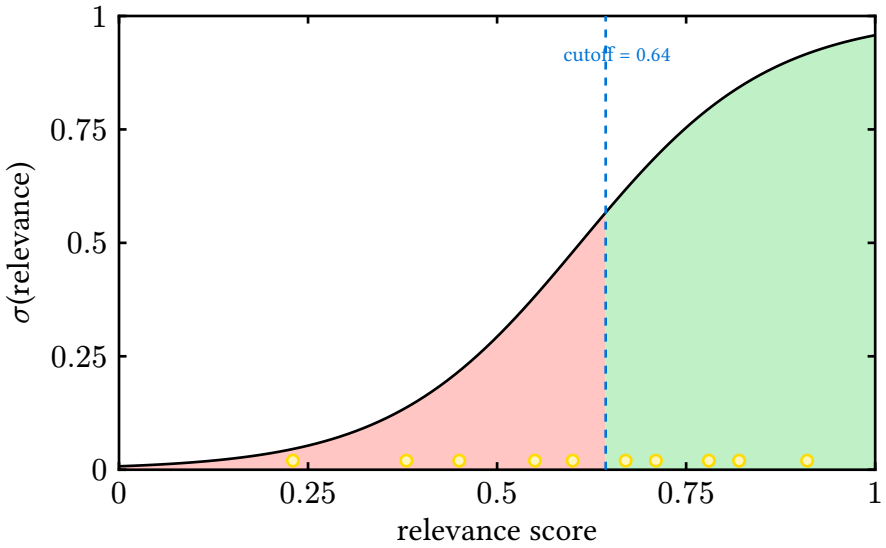


*Figure 2: Un esempio di funziona sigmoide utilizzata in fase di re-ranking per separare al meglio i chunk significativi da quelli non significativi*

Infine, i documenti selezionati vengono forniti come contesto al modello generativo, mantenendo informazioni circa la pertinenza con la query originale, per permettere all'agente di integrare in modo più efficace le informazioni durante la generazione della risposta.

## 3.4 Generazione e Validazione

Il modello generativo utilizza come contesto i documenti recuperati con il punteggio di rilevanza, la query originale e le relative hypothetical queries, per produrre una risposta contestualizzata e basata su fonti attendibili. La generazione della risposta è guidata da un prompt Jinja2, progettato per incoraggiare il modello a integrare le informazioni in modo coerente e a fornire spiegazioni sulle decisioni prese durante la generazione. L'uso di Jinja2 consente inoltre di strutturare dinamicamente il prompt, facilitando l'inserimento e la gestione di variabili (come documenti e query) in modo modulare, riutilizzabile e facilmente manutenibile. La risposta contiene citazioni ai documenti utilizzati, unito a un riassunto del contenuto e a una spiegazione del processo inferenziale seguito dal modello per arrivare alla risposta finale.

La fase di validazione dirige il flusso del sistema, determinando se iterare il processo di recupero delle informazioni o terminare la conversazione. In questo passaggio, l'agente fornisce una spiegazione testuale che descrive la logica alla base della propria decisione, migliorando la trasparenza del processo e consentendo all'utente di comprendere perché la risposta sia stata ritenuta soddisfacente o meno. In particolare, il modello assegna un

punteggio di soddisfazione compreso tra 0 e 1, calcolato secondo criteri predefiniti; una risposta è considerata accettabile quando il punteggio è ≥ 0,8. Se il punteggio è < 0,8, il sistema valuta se iterare il processo di recupero delle informazioni, aggiornando lo stato del grafo con le nuove informazioni ottenute e generando ulteriori query per affinare la ricerca, oppure se terminare la conversazione nel caso in cui ulteriori tentativi non siano probabilmente in grado di migliorare significativamente la risposta (ad esempio, a causa di documentazione insufficiente).

Un esempio completo dell'output del sistema, comprensivo dei chunk recuperati, dei punteggi di rilevanza e del ragionamento di validazione, è fornito nel Appendice.

# 4 Risultati

Sono stati condotti test preliminari per valutare l'efficacia del sistema RAG implementato, focalizzandosi sulla qualità del retrieval semantico, qualità delle risposte generate e tempi di ricerca. I risultati ottenuti indicano che il sistema è in grado di recuperare documenti rilevanti anche in presenza di query complesse o ambigue, grazie all'utilizzo di embedding semantici e alla generazione di hypothetical queries. Le risposte prodotte risultano contestualizzate e basate su fonti attendibili, con una riduzione significativa dei tempi di ricerca rispetto a un approccio basato esclusivamente manuale.

La componente più incisiva nella qualità del retrieval è stata la presenza di hypothetical queries: quasi sempre è bastato aggiungere la domanda giusta per recuperare correttamente il chunk più significativo. Secondariamente, un input ben strutturato dell'utente permette di fare una salto di qualità in questa fase. Per quanto riguarda la fase di generazione della risposta, è stata dedicata particolare attenzione al prompt engineering, poiché è emerso come il prompt rappresenti un fattore determinante per la qualità dell'output. A parità di un contesto informativo e di una query ben definiti, la struttura, la chiarezza e le istruzioni contenute nel prompt influenzano in modo significativo la coerenza, la completezza e l'affidabilità della risposta generata. Inoltre, una domanda chiara e precisa consente al modello di comprendere con maggiore accuratezza quali informazioni includere nella risposta, riducendo ambiguità e migliorando la pertinenza del risultato.

Per l'esecuzione dei test sono stati coinvolti operatori di AREU, che hanno fornito feedback qualitativi sulla pertinenza delle risposte e sulla facilità d'uso del sistema. Per automatizzare il processo di testing e disporre di un benchmark per la valutazione di sviluppi e miglioramenti futuri, è stato costruito un dataset di domande corredate dai contenuti attesi che devono essere presenti nelle rispettive risposte. I risultati preliminari suggeriscono un potenziale significativo per l'applicazione di sistemi RAG in contesti di emergenza, dove la rapidità e l'affidabilità delle informazioni sono cruciali.

Per valutare l'impatto della qualità del modello e ottimizzare l'uso delle risorse, è stato modificato esclusivamente il modello nel nodo responsabile della generazione della risposta. Negli altri nodi, invece, è stato mantenuto lo stesso modello, poiché task meno complessi non richiedono l'impiego di modelli più potenti e costosi. I tre modelli generativi confrontati sono: Amazon Nova 2 Lite, Claude Sonnet 4.6 e Mistral Pixtral Large. Come si evince da Figure 3 $A$, Sonnet è risultato il modello con le migliori prestazioni complessive,

distinguendosi per il più alto numero di risposte perfette; al contrario, Pixtral ha evidenziato le performance più deboli. Tuttavia, tra Sonnet e Nova le differenze sono state meno marcate. Questo suggerisce che la qualità del retrieval e la struttura del prompt possono avere un impatto più determinante sulla performance complessiva rispetto alla scelta specifica del modello generativo, almeno all'interno della gamma di modelli testati. Questo risultato evidenzia l'importanza di ottimizzare le componenti di retrieval e prompt engineering per massimizzare l'efficacia dei sistemi RAG, piuttosto che concentrarsi esclusivamente sulla selezione del modello generativo.

Per quanto riguarda i tempi di risposta, dalla Figure 3 $B$ si osserva una grande differenza in termini di latenza tra i tre modelli, con il modello Amazon Nova 2 Lite che genera le risposte in tempi significativamente inferiori rispetto a Claude Sonnet 4.6 e Mistral Pixtral Large. Questo risultato sottolinea l'importanza di considerare non solo la qualità delle risposte generate, ma anche l'efficienza computazionale dei modelli utilizzati, soprattutto in contesti come quello di AREU, dove la rapidità delle informazioni è cruciale.

Osservando più nel dettaglio i tempi negli altri nodi del grafo Figure 3 $C$, si è riscontrato che la fase di reranking rappresenta il collo di bottiglia principale, con tempi di esecuzione significativamente più elevati rispetto alle altre fasi. Questo risultato evidenzia l'importanza di ottimizzare la valutazione e selezione dei documenti recuperati, poiché rappresenta un punto critico per l'efficienza complessiva del sistema RAG. L'ottimizzazione di questa fase potrebbe comportare miglioramenti significativi nei tempi di risposta, rendendo il sistema più adatto a scenari reali in cui la rapidità delle informazioni è essenziale.

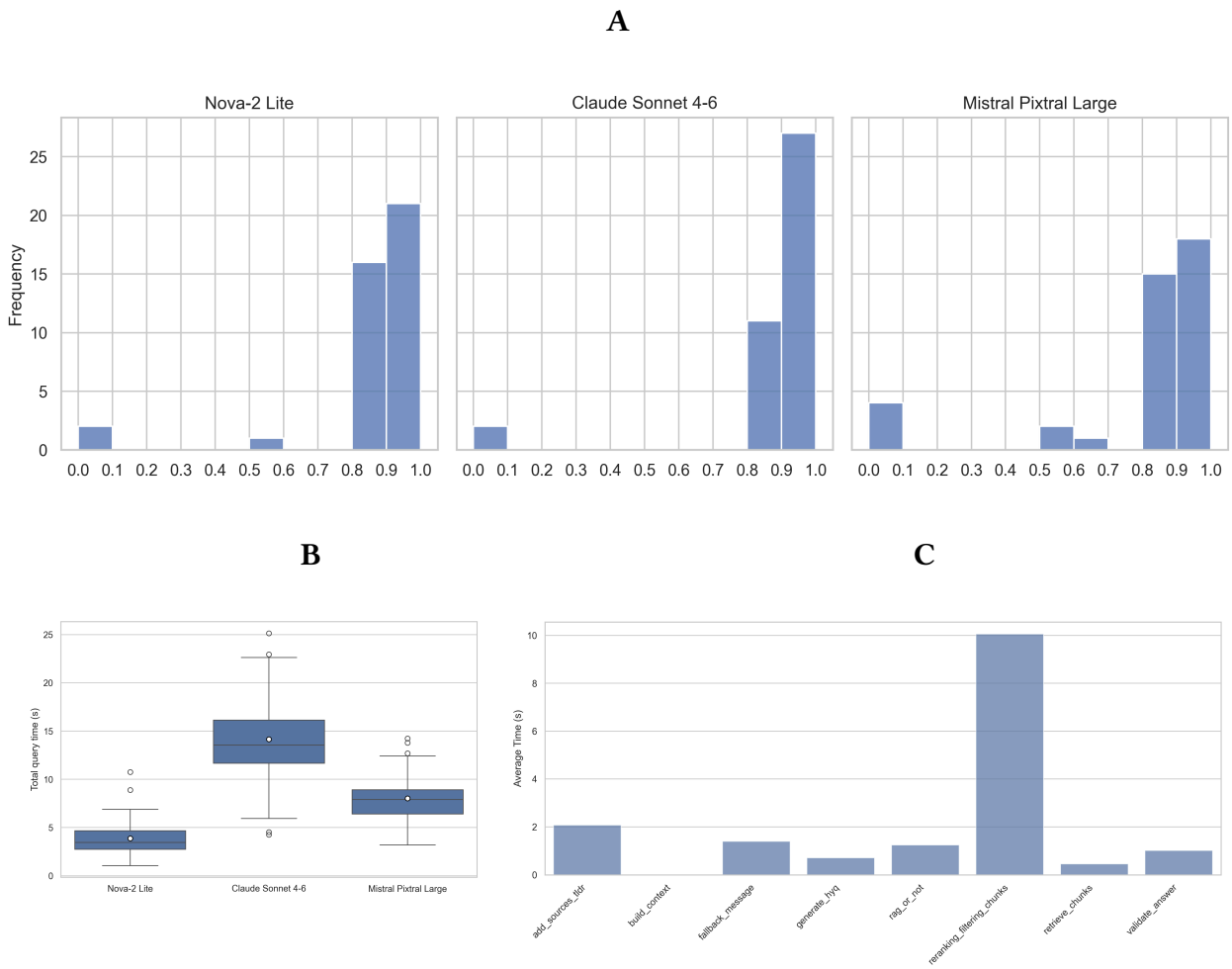


*Figure 3: Confronto tra punteggi e tempi*

*Il Pannello **A** mostra la distribuzione dei punteggi per ciascun modello. Possiamo osservare che Sonnet è il modello complessivamente migliore.*

*Il Pannello **B** mostra la distribuzione dei tempi di interrogazione per modello tramite un boxplot. Nova-2-Lite ha la latenza più bassa, in linea con le sue dimensioni ridotte.*

*Il Pannello **C** mostra il tempo medio di esecuzione per ciascun nodo della pipeline. Il collo di bottiglia è rappresentato dalla fase di re-ranking, che richiede un tempo significativamente maggiore rispetto alle altre fasi, suggerendo che l'ottimizzazione di questo passaggio potrebbe portare a miglioramenti sostanziali delle prestazioni complessive del sistema.*

# 5 Discussione

Il sistema RAG implementato ha dimostrato di essere efficace nel recuperare informazioni rilevanti e nel generare risposte contestualizzate, migliorando l'affidabilità e la pertinenza delle informazioni fornite agli operatori di AREU. Tuttavia, sono emerse alcune limitazioni, tra cui la necessità di ulteriori ottimizzazioni per gestire scenari più complessi o dinamici. Nello specifico, i documenti utilizzati per il retrieval non sono stati prodotti con l'obiettivo di essere processati da un sistema RAG, e presentano quindi una struttura non ottimale per il processo di retrieval e generazione. In particolare, la presenza di informazioni ridondanti (stessi concetti ripetuti in diverse sezioni dello stesso documento o distribuiti tra documenti differenti), obsolete (direttive non aggiornate) o non strutturate (immagini e tabelle) ha reso più difficile per il sistema identificare e integrare le informazioni più rilevanti, evidenziando l'importanza di un processo di cura e organizzazione dei dati più mirato per massimizzare l'efficacia dei sistemi RAG. Avere dei documenti progettati fin dall'inizio per un sistema RAG, permette di ridurre il numero di chunk da inviare al nodo di reranking, ottimizzando così le performance complessive, poiché il sistema impiega meno tempo sia per processarli sia per effettuare il ragionamento sui contenuti.

Tra i punti di forza del sistema spicca la modularità offerta dal framework DecAgents e LangGraph. Questo approccio multi-agente, supportato dalla libreria di networking production-ready di Ottante S.r.l., consente di aggiornare o sostituire i singoli componenti senza influire sul flusso di lavoro complessivo. Inoltre, la capacità di integrare dinamicamente conoscenze esterne migliora la fattualità e la trasparenza delle risposte generate. L'adozione di un'architettura basata su grafi ha inoltre permesso l'implementazione di strategie di reasoning più avanzate e il mantenimento di una traccia completa delle interazioni, facilitando sia il raffinamento iterativo sia l'analisi post-hoc delle prestazioni del sistema.

Tra gli sviluppi futuri, si prevede di esplorare l'integrazione con sistemi sanitari esistenti, al fine di migliorare ulteriormente la pertinenza e l'utilità delle risposte fornite agli operatori di AREU. Un'altra direzione di ricerca riguarda l'esplorazione della multimodalità, integrando fonti di conoscenza non testuali come immagini o dati strutturati per arricchire ulteriormente il contesto fornito al modello generativo. Infine, si prevede di condurre un confronto sistematico tra diversi modelli di embedding e generazione, al fine di identificare le configurazioni più efficaci per il contesto specifico di AREU, e di esplorare strategie di

hierarchical ranking per ottimizzare ulteriormente la selezione dei documenti durante il processo di retrieval.

In conclusione, un sistema RAG si rivela particolarmente prezioso in situazioni di emergenza, poiché permette di aumentare l'efficienza operativa, garantendo che le risposte siano sempre basate su documenti attendibili. La flessibilità e l'adattabilità del sistema lo rendono utilizzabile in molteplici scenari, supportando diversi professionisti, dai soccorritori agli operatori sanitari, fino ai responsabili della logistica e delle comunicazioni, contribuendo così a decisioni più rapide e informate.

# Bibliography


[Lew+20] P. Lewis *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. [Online]. Available: https://arxiv.org/abs/2005.11401

[Vas+17] A. Vaswani *et al.*, "Attention Is All You Need," *arXiv preprint arXiv:1706.03762*, 2017, [Online]. Available: https://arxiv.org/abs/1706.03762

[Gao+23] Y. Gao *et al.*, "Retrieval-Augmented Generation for Large Language Models: A Survey," *arXiv preprint arXiv:2312.10997*, 2023, [Online]. Available: https://arxiv.org/abs/2312.10997

[Rob+94] S. E. Robertson, S. Walker, S. Jones, M. Hancock-Beaulieu, and M. Gatford, "Okapi at TREC-3," *Proceedings of The Third Text REtrieval Conference (TREC 1994)*, 1994, [Online]. Available: https://www.researchgate.net/publication/221037764_Okapi_at_TREC-3

[Che+25] M. Cheng *et al.*, "A Survey on Knowledge-Oriented Retrieval-Augmented Generation," *arXiv preprint arXiv:2503.10677*, 2025, [Online]. Available: https://arxiv.org/abs/2503.10677

[ZC25] J. Zhou and L. Chen, "OpenRAG: Optimizing Retrieval-Augmented Generation End-to-End via In-Context Retrieval Learning," *arXiv preprint arXiv:2503.08398*, 2025, [Online]. Available: https://arxiv.org/abs/2503.08398

[Cuc+24] F. Cuconasu *et al.*, "The Power of Noise: Redefining Retrieval for Retrieval-Augmented Generation," *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2024, [Online]. Available: https://arxiv.org/abs/2401.14887

# Appendice

Il seguente estratto JSON illustra la struttura del dataset di valutazione utilizzato per analizzare le prestazioni del sistema. Ogni voce rappresenta una singola domanda di test insieme alla relativa risposta attesa, alla risposta generata dal sistema, alla traccia di ragionamento prodotta durante la fase di recupero delle informazioni, ai documenti di riferimento identificati come rilevanti, ai chunk recuperati con i rispettivi punteggi di rilevanza, alla suddivisione dei tempi per ciascun nodo della pipeline e all'esito della valutazione automatizzata. Il dataset è composto da 40 domande derivate dalle procedure operative di AREU ed è stato elaborato utilizzando la configurazione del modello Nova-2-Lite. Per ragioni di brevità, qui viene riportato solo un campione rappresentativo.


```
{
  "run_info": {
      "timestamp": "2026-04-02 12:48:55 CEST",
      "agent_config": "eu.amazon.nova-2-lite-v1:0",
      "language": "Italiano",
      "num_questions": 40
  },
  "questions": [
      {
          "id_question": "1",
          "question": "L'operatore tecnico che processa richiesta di soccorso
può declassare in autonomia il codice evento da rosso a giallo?",
          "expected_answer": "Sì - L'operatore tecnico può declassare in
autonomia un codice da rosso a giallo motivando la scelta nelle note
strutturate",
          "answer": "Sì, l'operatore tecnico che processa la richiesta di
soccorso può declassare in autonomia il codice evento da rosso a giallo, a
condizione che venga esplicitata la motivazione nel campo Note. Questa
possibilità è regolamentata specificamente dalle indicazioni contenute nel
documento operativo, che stabilisce che non ci sono limitazioni al
declassamento del codice di gravità da rosso a giallo, a differenza di altri
declassamenti che richiedono una rivalutazione sanitaria.",
          "reasoning": "L'utente chiede se l'operatore tecnico che gestisce le
richieste di soccorso può declassare autonomamente il codice evento da rosso a
giallo. Il contesto fornisce informazioni chiare al riguardo. Il documento IOP
12 Risposta alla chiamata-filtro-rifiuto socc-revoca Rev9 del 02-08-2021.pdf
stabilisce che non ci sono limitazioni al declassamento del codice di gravità
```

da rosso a giallo, purché venga esplicitata la motivazione nel campo 'Note'. Questo documento è il più rilevante e fornisce una risposta diretta alla domanda dell'utente. Gli altri documenti forniscono contesto aggiuntivo sui codici di gravità e sui ruoli, ma non cambiano la conclusione principale.",
          "tldr": "Puoi declassare un codice evento da rosso a giallo in autonomia, basta che scrivi la motivazione nel campo Note. Questo è permesso dal documento operativo e non serve una rivalutazione sanitaria, a differenza di altri declassamenti.",
          "reference_documents": [
              "IOP 12 Risposta alla chiamata-filtro-rifiuto socc-revoca Rev9 del 02-08-2021.pdf",
              "DOC 12 Utilizzo applicativo informatico SOREU Rev3 del 12-06-2020.pdf",
              "DOC 12 Utilizzo applicativo informatico SOREU Rev3 del 12-06-2020 con modifiche in evidenza.pdf",
              "DOC 109 Attività consolle flotta e sanitaria Rev1 del 14-07-2015.pdf"
          ],
          "retrieved_chunks": [
              {
                  "chunk_id": "0d3ab50c-302f-4d90-b8f6-78e84e812421_20",
                  "doc_id": 6,
                  "filename": "IOP 12 Risposta alla chiamata-filtro-rifiuto socc-revoca Rev9 del 02-08-2021.pdf",
                  "chunk_reasoning": "Questo frammento è direttamente pertinente alla domanda poiché stabilisce chiaramente che l'operatore tecnico può declassare il codice da rosso a giallo senza necessità di rivalutazione sanitaria, a patto di motivare la decisione nel campo 'Note'.",
                  "chunk_relevance": 1.0,
                  "above_threshold": true
              },
              {
                  "chunk_id": "415f1739-51cb-4835-ba2a-f1ffceba0477_24",
                  "doc_id": 84,
                  "filename": "DOC 12 Utilizzo applicativo informatico SOREU Rev3 del 12-06-2020.pdf",
                  "chunk_reasoning": "Questo frammento è rilevante perché descrive i codici di gravità degli eventi e menziona che l'operatore sanitario può modificare il codice dopo una rivalutazione, fornendo un contesto utile per capire le possibilità di declassamento.",
                  "chunk_relevance": 0.8,
                  "above_threshold": true

```
            },
            {
                "chunk_id": "e79b3cc3-39ee-4adb-82e6-6367e0081479_24",
                "doc_id": 83,
                "filename": "DOC 12 Utilizzo applicativo informatico SOREU
Rev3 del 12-06-2020 con modifiche in evidenza.pdf",
                "chunk_reasoning": "Questo frammento ripete le informazioni
sui codici di gravità e menziona che l'operatore sanitario può modificare il
codice dopo una rivalutazione, fornendo ulteriore contesto ma non risolvendo
direttamente la domanda sull'autonomia dell'operatore tecnico.",
                "chunk_relevance": 0.8,
                "above_threshold": true
            },
            {
                "chunk_id": "0b51feec-ea88-40a6-8f6f-f16cb8274fdd_23",
                "doc_id": 83,
                "filename": "DOC 12 Utilizzo applicativo informatico SOREU
Rev3 del 12-06-2020 con modifiche in evidenza.pdf",
                "chunk_reasoning": "Questo frammento descrive i codici
colore e il ruolo dell'operatore tecnico receiver nel definire il codice di
gravità, supportando il contesto della domanda ma non fornendo informazioni
specifiche sul declassamento autonomo da rosso a giallo.",
                "chunk_relevance": 0.7,
                "above_threshold": true
            },
            {
                "chunk_id": "1da9db4b-1b49-49a7-9648-268cef0b7ce6_28",
                "doc_id": 75,
                "filename": "DOC 109 Attività consolle flotta e sanitaria
Rev1 del 14-07-2015.pdf",
                "chunk_reasoning": "Questo frammento indica che la consolle
sanitaria può ridefinire il codice di gravità in qualsiasi momento, ma non
specifica se l'operatore tecnico può farlo autonomamente, quindi è meno
direttamente rilevante della chunk 0d3ab50c-302f-4d90-b8f6-78e84e812421_20.",
                "chunk_relevance": 0.6,
                "above_threshold": true
            },
            {
                "chunk_id": "695b709a-1417-4fc5-9051-6d9881dcbce6_26",
                "doc_id": 83,
                "filename": "DOC 12 Utilizzo applicativo informatico SOREU
Rev3 del 12-06-2020 con modifiche in evidenza.pdf",
```

"chunk_reasoning": "Questo frammento spiega come l'operatore dispatcher può modificare i codici di invio del mezzo, ma non affronta direttamente la possibilità di declassamento del codice evento da parte dell'operatore tecnico, quindi è meno rilevante.",
"chunk_relevance": 0.5,
"above_threshold": false
},
{
"chunk_id": "60d89164-2741-46fa-b5bf-536677cc87cd_26",
"doc_id": 84,
"filename": "DOC 12 Utilizzo applicativo informatico SOREU Rev3 del 12-06-2020.pdf",
"chunk_reasoning": "Questo frammento descrive le azioni dell'operatore dispatcher in caso di codici evento bianchi o verde/giallo, ma non affronta il declassamento da rosso a giallo da parte dell'operatore tecnico, quindi ha una rilevanza moderata.",
"chunk_relevance": 0.5,
"above_threshold": false
},
{
"chunk_id": "9a5c9153-f577-487d-89e5-36bc9aa814d9_63",
"doc_id": 6,
"filename": "IOP 12 Risposta alla chiamata-filtro-rifiuto socc-revoca Rev9 del 02-08-2021.pdf",
"chunk_reasoning": "Questo frammento descrive le procedure per la revoca di una richiesta di soccorso e non è direttamente rilevante per la domanda sul declassamento del codice evento, ma fornisce contesto aggiuntivo sulle operazioni dell'operatore tecnico.",
"chunk_relevance": 0.4,
"above_threshold": false
},
{
"chunk_id": "553f7297-0d5d-4ebe-87eb-b995476691ed_28",
"doc_id": 108,
"filename": "DOC 161 Riscontro sanitario SOREU Rev1 del 08-01-2026.pdf",
"chunk_reasoning": "Questo frammento spiega che il codice di primo riscontro può essere variato dal personale sanitario in funzione di aspetti logistici, ma non si riferisce all'operatore tecnico o al declassamento da rosso a giallo, quindi ha una bassa rilevanza.",
"chunk_relevance": 0.4,
"above_threshold": false

```
            },
            {
                "chunk_id": "93546983-ca96-4290-bf96-e77ddea3eeb2_27",
                "doc_id": 83,
                "filename": "DOC 12 Utilizzo applicativo informatico SOREU
Rev3 del 12-06-2020 con modifiche in evidenza.pdf",
                "chunk_reasoning": "Questo frammento si concentra sui codici
di riscontro in posto e non è direttamente rilevante per la domanda sul
declassamento del codice evento da parte dell'operatore tecnico, quindi ha una
bassa rilevanza.",
                "chunk_relevance": 0.3,
                "above_threshold": false
            },
            {
                "chunk_id": "4ef1cedf-ad0c-4c0f-912d-aee7a2b6e1dc_36",
                "doc_id": 108,
                "filename": "DOC 161 Riscontro sanitario SOREU Rev1 del
08-01-2026.pdf",
                "chunk_reasoning": "Questo frammento riguarda i pazienti con
patologie tempo-dipendenti e i codici gialli/rossi associati, ma non è
direttamente rilevante per la domanda sul declassamento autonomo del codice
evento da parte dell'operatore tecnico.",
                "chunk_relevance": 0.3,
                "above_threshold": false
            },
            {
                "chunk_id": "529007cc-7f9c-4f22-8521-fcb86934c155_13",
                "doc_id": 115,
                "filename": "DOC 170 Rivalutazione codici non urgenti Rev0
del 31-05-2024.pdf",
                "chunk_reasoning": "Questo frammento descrive le azioni da
intraprendere da parte dell'infermiere in caso di richiesta di invio di un MSB
e modifica del codice di gravità, ma non si riferisce specificamente al
declassamento da rosso a giallo da parte dell'operatore tecnico, quindi ha una
bassa rilevanza.",
                "chunk_relevance": 0.3,
                "above_threshold": false
            },
            {
                "chunk_id": "a6e4539d-88c2-4d7f-9f74-b1aff9ede70c_17",
                "doc_id": 115,
                "filename": "DOC 170 Rivalutazione codici non urgenti Rev0
```

del 31-05-2024.pdf",
                "chunk_reasoning": "Questo frammento tratta la gestione della revoca del paziente e non è direttamente rilevante per la domanda sul declassamento del codice evento, quindi ha una bassa rilevanza.",
                "chunk_relevance": 0.3,
                "above_threshold": false
            },
            {
                "chunk_id": "ecb34f68-c440-490b-9297-3a570c43b9d7_31",
                "doc_id": 108,
                "filename": "DOC 161 Riscontro sanitario SOREU Rev1 del 08-01-2026.pdf",
                "chunk_reasoning": "Questo frammento definisce i significati dei codici giallo, rosso e nero per i pazienti, ma non fornisce informazioni sulle procedure di declassamento da parte dell'operatore tecnico, quindi ha una rilevanza molto bassa.",
                "chunk_relevance": 0.2,
                "above_threshold": false
            },
            {
                "chunk_id": "055023cc-06f8-4d6c-8043-d8f563080040_11",
                "doc_id": 192,
                "filename": "PRO 21 Gestione regionale mezzi di soccorso Rev6 del 16-01-2024.pdf",
                "chunk_reasoning": "Questo frammento descrive la modalità di selezione del mezzo condiviso e non è rilevante per la domanda sul declassamento del codice evento, quindi ha una rilevanza molto bassa.",
                "chunk_relevance": 0.2,
                "above_threshold": false
            },
            {
                "chunk_id": "674d05d3-2766-4c8d-8917-1bb984f23214_14",
                "doc_id": 193,
                "filename": "PRO 22 Gestione maxiemergenze Rev5 del 14-07-2025.pdf",
                "chunk_reasoning": "Questo frammento riguarda la gestione delle maxiemergenze e il ruolo del Referente Tecnico di Turno, ma non affronta il declassamento del codice evento da parte dell'operatore tecnico, quindi ha una rilevanza molto bassa.",
                "chunk_relevance": 0.1,
                "above_threshold": false
            }

```
            ],
            "thread_id": "areu_batch_1_20260402_124855",
            "timings": {
                "node_times": {
                    "rag_or_not": 1.574,
                    "generate_hyq": 0.742,
                    "retrieve_chunks": 0.63,
                    "reranking_filtering_chunks": 10.594,
                    "build_context": 0.0,
                    "gen_rag_answer": 2.873,
                    "validate_answer": 0.994,
                    "add_sources_tldr": 1.798
                },
                "rag_invoke_time_s": 19.283,
                "check_time_s": 1.27,
                "total_time_s": 20.553
            },
            "evaluation": {
                "check_passed": true,
                "check_correspondence_score": 0.9,
                "check_missing_points": "none",
                "check_reasoning": "La risposta generata copre tutti i punti
chiave della risposta attesa: conferma che l'operatore tecnico può declassare
il codice da rosso a giallo in autonomia e l'obbligo di motivare la scelta
nelle note. L'informazione aggiuntiva sul documento operativo e sulle
limitazioni per altri declassamenti è corretta e non contraddice la risposta
attesa, quindi non influisce negativamente sulla corrispondenza."
            }
        },
        ...
    ]
}
```